

MecâniQA: Descobrimo o Desenvolvimento Web Orientado a Objetos

Autores: Darlan Deivid Santos Silva
Dylan Coêlho Soares
Gustavo Costa Pereira
Hudnei Sued Passos Santana
Leonardo Freitas de Carvalho Nery

Agenda

O objetivo desta apresentação é apresentar a arquitetura do MVP da MecaniQA, mostrando na prática como os fundamentos de Orientação a Objetos sustentam a construção de uma API RESTful.

1. Contexto Inicial:

Apresentando a empresa, o desafio proposto e os objetivos do nosso mvp.

2. Conceitos de OO:

Definição do paradigma de Orientação a objetos e a explicação dos seus conceitos básicos.

3. Construindo APIs Rest com controllers:

Rotas, Endpoints, Verbos e Status em HTTP.

4. Gerenciando Estado em memória:

Implementação com Membros estáticos e o Padrão Singleton.

5. Validando a API:

Resultado dos testes e endpoints obtidos.

7. Referências:

Resumo da nossa experiência com o projeto.

Contexto Inicial



A MecâniQA é uma empresa baiana de tecnologia voltada para o setor automotivo

Estão migrando seu sistema legado para WEB com a ajuda da equipe de trainees.

A primeira etapa visa Desenvolver uma Web API em Java (Spring Boot).



Introdução à OO: Classes e Atributos



Classes: Moldes que definem as propriedades e os comportamentos de um objeto.

Atributos: Dados ou características que descrevem esse objeto.



```
public class Servico {  
  
    private long id;  
    private String nomeServico;  
    private int tempoEstimado;  
    private double custoTabelado;  
    private LocalDateTime dataCadastro;  
    private LocalDateTime dataUltimaAtualizacao;  
}
```

Introdução à OO: Construtor

Um construtor é a função responsável por criar e inicializar um objeto na memória.

Em silêncio (não possui retorno),
ele pega as informações do
carro e o fábrica.



Introdução à OO: Enums

Enums são um tipo especial de dados usados para definir um conjunto fixo de constantes, como uma lista fechada de opções que não mudam.

```
public enum CategoriaPeca {  
  
    MOTOR,  
    SUSPENSAO,  
    FREIOS,  
    ELETRICA,  
    ACESSORIOS  
  
}
```

Introdução à OO: Modificadores de Acesso

public: Acesso livre. Qualquer parte do programa, de dentro ou de fora da classe, pode acessar o atributo ou método.

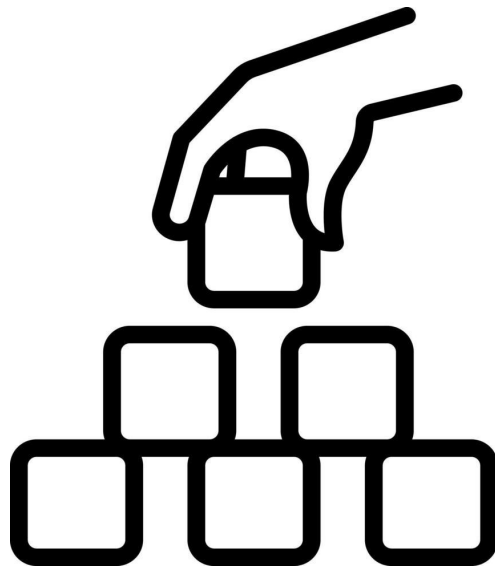
private: Acesso restrito. Apenas a própria classe que criou o membro pode vê-lo ou modificá-lo. Nenhuma outra classe tem acesso.



Introdução à OO: Getter e Setter

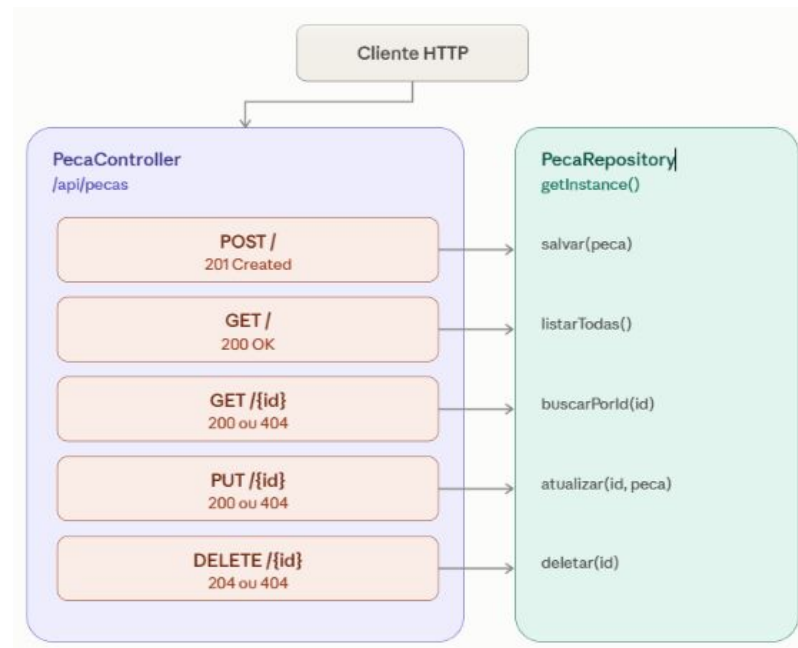
Getter (Leitura): Método público utilizado para acessar e retornar o valor de um atributo privado.

Setter (Modificação): Método público utilizado para alterar ou definir o valor de um atributo privado, permitindo aplicar regras e validações antes de atualizar o dado.



Construindo uma Api Rest: Controllers

O Controller é a porta de entrada da API. Ele recebe as requisições da web, chama as regras de negócio e devolve as respostas.



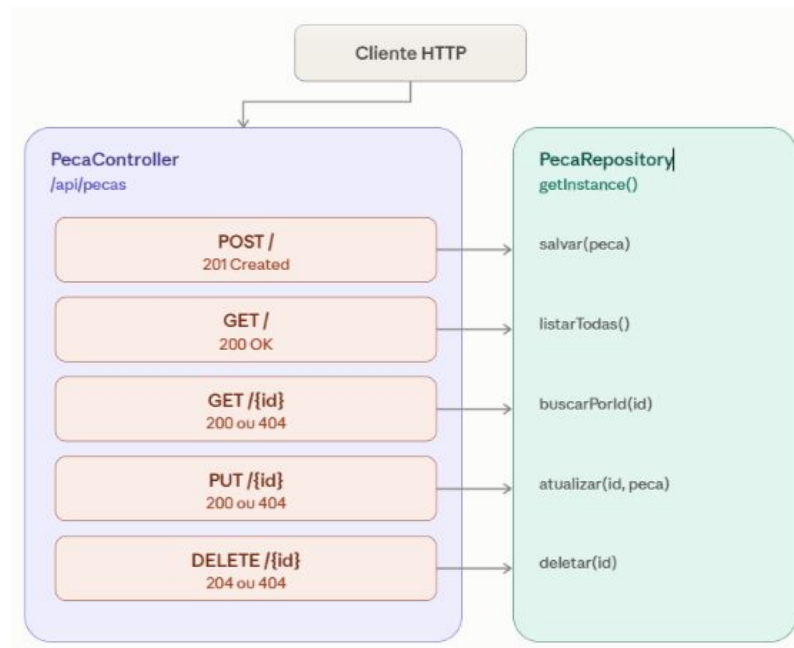
Construindo uma Api Rest: Verbos e Status

@PostMapping: acionado para criar novo registro

@GetMapping: serve para buscar e listar os dados.

@PutMapping: serve para atualizar dados

@DeleteMapping: serve para deletar dados



Atributos Static

Static: são atributos que pertencem a classe como um todo, e não somente a um objeto, como por exemplo o total de carros fabricados por uma montadora.



Gerenciamento de Estado em Memória: Padrão Singleton



O Singleton é um padrão de projeto criacional que garante que uma classe tenha apenas uma única instância em toda a aplicação e fornece um ponto de acesso global para ela

A code editor window with a white background and a light gray border. It has three colored window control buttons (red, yellow, green) in the top left corner. The code is written in a syntax-highlighted style with various colors for keywords, identifiers, and literals.

```
public static PecaRepository getInstance()  
{  
    if (instancia == null){  
        instancia = new PecaRepository();  
    }  
    return instancia;  
}
```

Validando a API: Resultados dos testes

The screenshot displays a REST client interface with a dark theme. At the top, a green bar indicates a **POST** request to `http://localhost:8080/api/servicos`, which was successfully executed, returning a **201 Created** status. The response time is **469 ms** and the body size is **192 B**. Below this, the **Body** tab is selected, showing the request payload in JSON format:

```
1 {
2   "nomeServico": "Alinhamento e Balanceamento",
3   "tempoEstimado": 60,
4   "custoTabelado": 250.00
5 }
```

To the right, the **Preview** tab shows the response payload in JSON format:

```
1 {
2   "id": 1,
3   "nomeServico": "Alinhamento e Balanceamento",
4   "tempoEstimado": 60,
5   "custoTabelado": 250.0,
6   "dataCadastro": "2026-08-29T21:24:47.6082246",
7   "dataUltimaAtualizacao": "2026-08-29T21:24:47.6082246"
8 }
```

Validando a API: Resultados dos testes

The screenshot displays a REST client interface with a dark theme. At the top, a green tab indicates a **POST** request to `http://localhost:8080/api/servicos`. A purple **Send** button is visible. To the right, a green status bar shows **201 Created**, with response times of **44 ms** and size of **178 B**. The timestamp **Just Now** is on the far right.

Below the status bar, a row of tabs includes **Params**, **Body** (selected with a green dot), **Auth**, **Headers** (4), **Scripts**, and **Docs**. On the right side of this row are **Preview**, **Headers** (3), **Cookies**, **Tests** (0/0), **→ Mock**, and **Console**.

The **Body** tab on the left shows a JSON object:

```
1 {  
2   "nomeServico": "Troca de Óleo",  
3   "tempoEstimado": 45,  
4   "custoTabelado": 150.00  
5 }
```

The **Preview** tab on the right shows the response JSON:

```
1 {  
2   "id": 2,  
3   "nomeServico": "Troca de Óleo",  
4   "tempoEstimado": 45,  
5   "custoTabelado": 150.0,  
6   "dataCadastro": "2026-08-29T21:25:53.2599477",  
7   "dataUltimaAtualizacao": "2026-08-29T21:25:53.261023"  
8 }
```

At the bottom right, a small inset window shows the raw response body in JSON format:

```
1 {  
2   "nomeServico": "Troca de Óleo",  
3   "tempoEstimado": 45,  
4   "custoTabelado": 150.00  
5 }
```

Referências



- SPRING. Building an Application with Spring Boot. Disponível em: <https://spring.io/guides/gs/spring-boot>. Acesso em: 26 ago. 2026.
- W3SCHOOLS. Java OOP (Object-Oriented Programming). Disponível em: https://www.w3schools.com/java/java_oop.asp. Acesso em: 29 ago. 2026.
- REFACTORING.GURU. Singleton. Disponível em: <https://refactoring.guru/pt-br/design-patterns/singleton>. Acesso em: 29 ago. 2026.